

PAPOP v5.0.2

Strict Approval

Windows 图文安装与使用指南

适用于 Zcode • ChatGPT / Codex • DeepSeek Harness

用户级规则 GlobalRules	项目级规则 ProjectRules	治理模式 Strict Approval
----------------------	-----------------------	-------------------------

本指南基于 PAPOP v5.0.2 Strict Approval 发布包。安装位置以 Windows 为主；Codex 与 DeepSeek Harness 的用户目录可被 CODEX_HOME / DSH_HOME 环境变量改写。

版本日期：2026-09-17

1. 一页看懂：你实际上只需要做两次复制

PAPOP v5 的安装分成两层：GlobalRules 负责所有任务都应该遵守的行为边界；ProjectRules 负责具体工作区里的状态机、账本、审批、Checkpoint 与恢复。Strict Approval 包已经同时包含这两层。

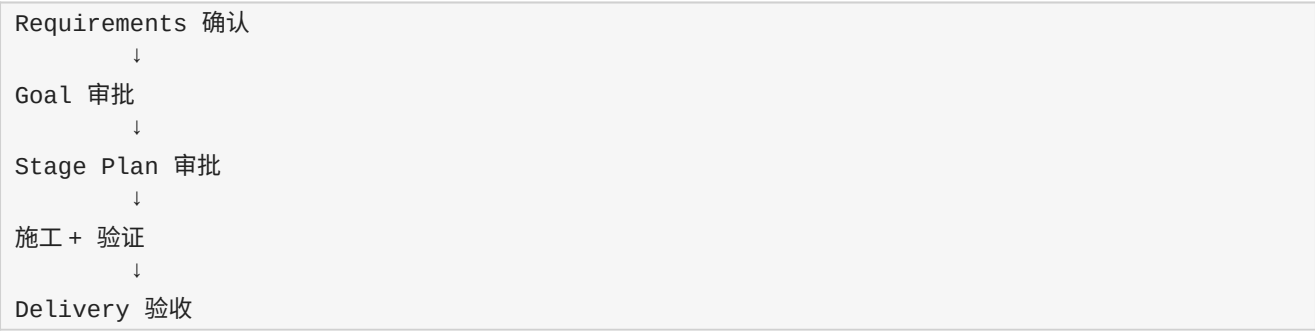
Harness	GlobalRules 目标位置 (Windows 默认)	ProjectRules 目标位置
Zcode	C:\Users\<你的用户名>\.zcode\	工作区根目录
ChatGPT / Codex	C:\Users\<你的用户名>\.codex\	工作区根目录
DeepSeek Harness	C:\Users\<你的用户名>\.dsh\	工作区根目录

重要：三个发布包只能选一个。Strict Approval 与 Autonomous 不要同时复制到同一个工作区。若已有自己的 AGENTS.md 或其他规则文件，先备份并人工合并，不要无脑覆盖。

安装总流程

- 1. 从 v5.0.2 Release 下载 portable-agent-project-operating-protocol-v5.0.2-strict-approval.zip。
- 2. 完整解压，确认点号开头的隐藏目录也存在。
- 3. 打开 GlobalRules，把里面的全部内容复制到你正在使用的 Harness 用户级规则目录。
- 4. 打开 ProjectRules，把“里面的全部内容”复制到目标工作区根目录，不要把 ProjectRules 外壳一起套进去。
- 5. 重新开启一个全新 Session，让 Agent 实际检查它加载了哪些规则，再开始正式任务。

Strict Approval 的核心流程



Strict Approval 不是“每条命令都问一次”。你审批的是需求、最终目标、当前 Stage 的计划和交付；已经批准的 Plan 内，普通施工步骤可以连续执行。

2. 下载 v5.0.2 Strict Approval 发布包

打开 PAPOP 的 Releases 页面，找到 v5.0.2，在 Assets 中下载 Strict Approval ZIP：

`portable-agent-project-operating-protocol-v5.0.2-strict-approval.zip`

Release 页面：<https://github.com/canpus/portable-agent-project-operating-protocol/releases>



图 1 v5.0.2 Release：选择 strict-approval ZIP，而不是 autonomous 或 global-rules-only。

如果你只想要“行为规则”而不需要项目状态机，可以选 GlobalRules Only；如果你希望 Agent 在授权范围内持续推进，则选 Autonomous。本文后续全部按 Strict Approval 编写。

解压后应该看到什么

- GlobalRules：用户级规则。
- ProjectRules：工作区级规则与状态机入口。
- docs、README、CHANGELOG 等说明文件。

Windows 可能隐藏以 “.” 开头的目录。复制时必须确认 .agent-rules / .agent-protocol 等隐藏目录没有被漏掉。

3. 安装 GlobalRules：先找到你的 Windows 用户目录

GlobalRules 要放到“当前 Harness 的用户级规则目录”。在 Windows 上，最简单的路径是先进入 C 盘，再进入 Users（资源管理器中文界面通常显示为“用户”），最后打开与你当前 Windows 用户名相同的文件夹。

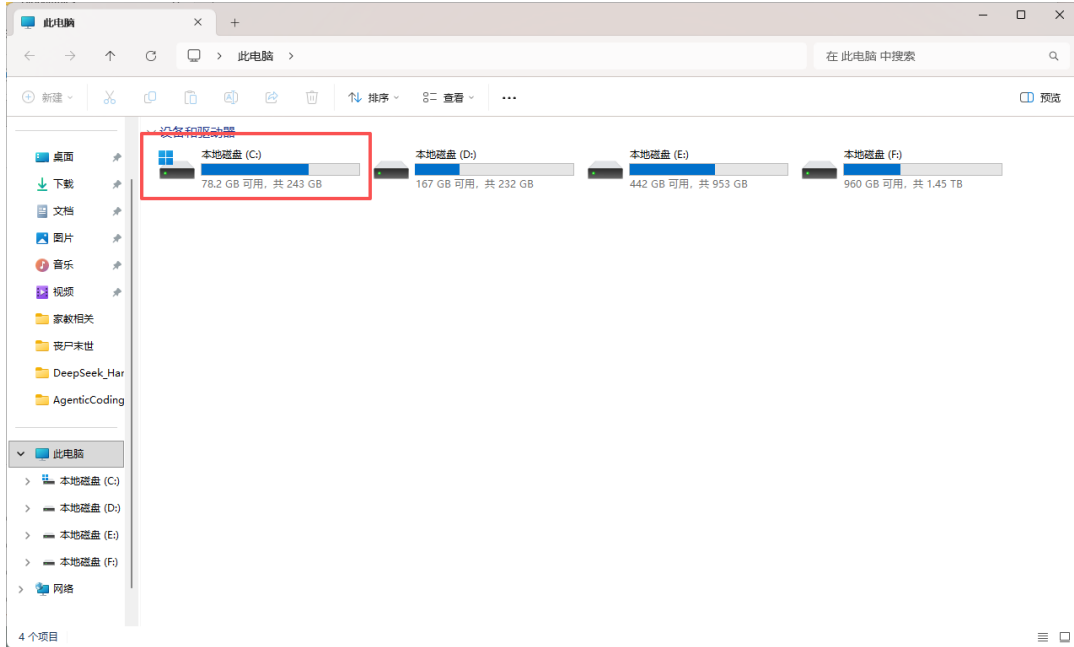


图 2 先打开系统盘（通常是 C:）。

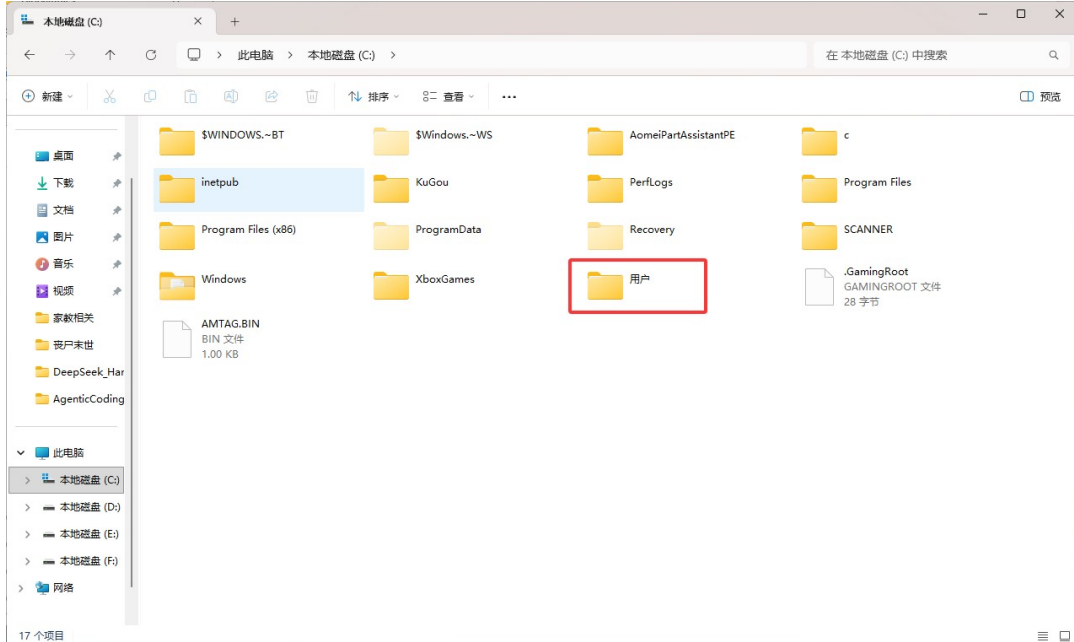


图 3 在 C: 根目录打开“用户 / Users”。

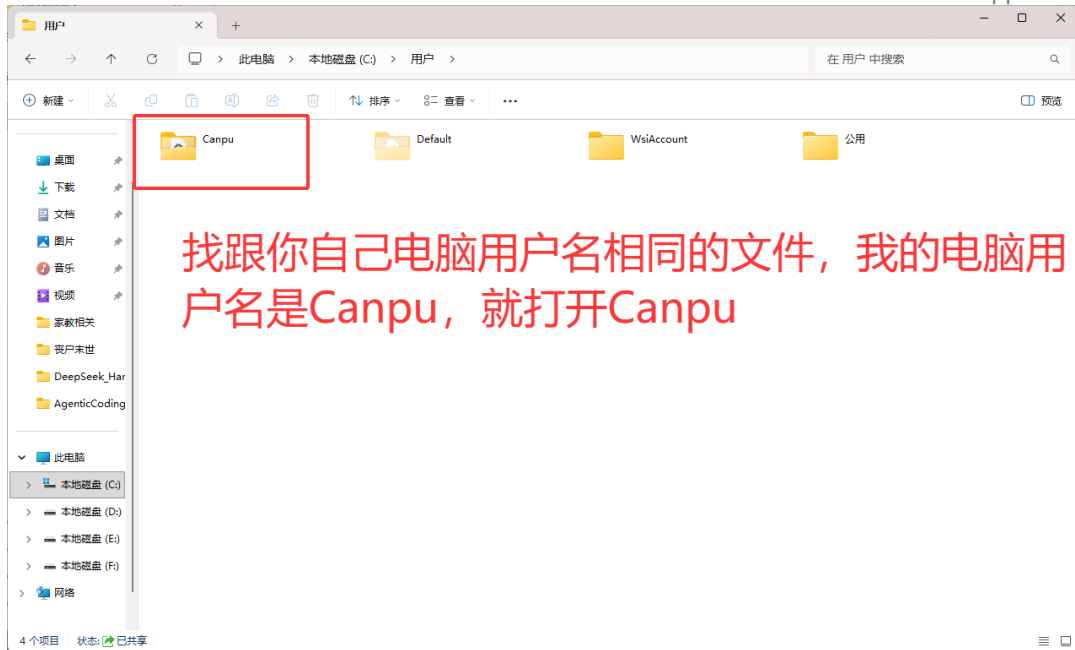


图 4 找到与你的 Windows 用户名相同的目录；示例用户名为 Canpu。

C:\Users\<你的用户名>\

“C:\用户\...” 是资源管理器的显示方式；实际文件系统路径通常仍是 C:\Users\....

4. 三种 Harness 的 GlobalRules 安装位置

进入 Windows 用户目录后，你可能会同时看到 .zcode、.codex、.dsh 等点号开头的目录。它们分别对应不同 Harness 的用户级配置根。

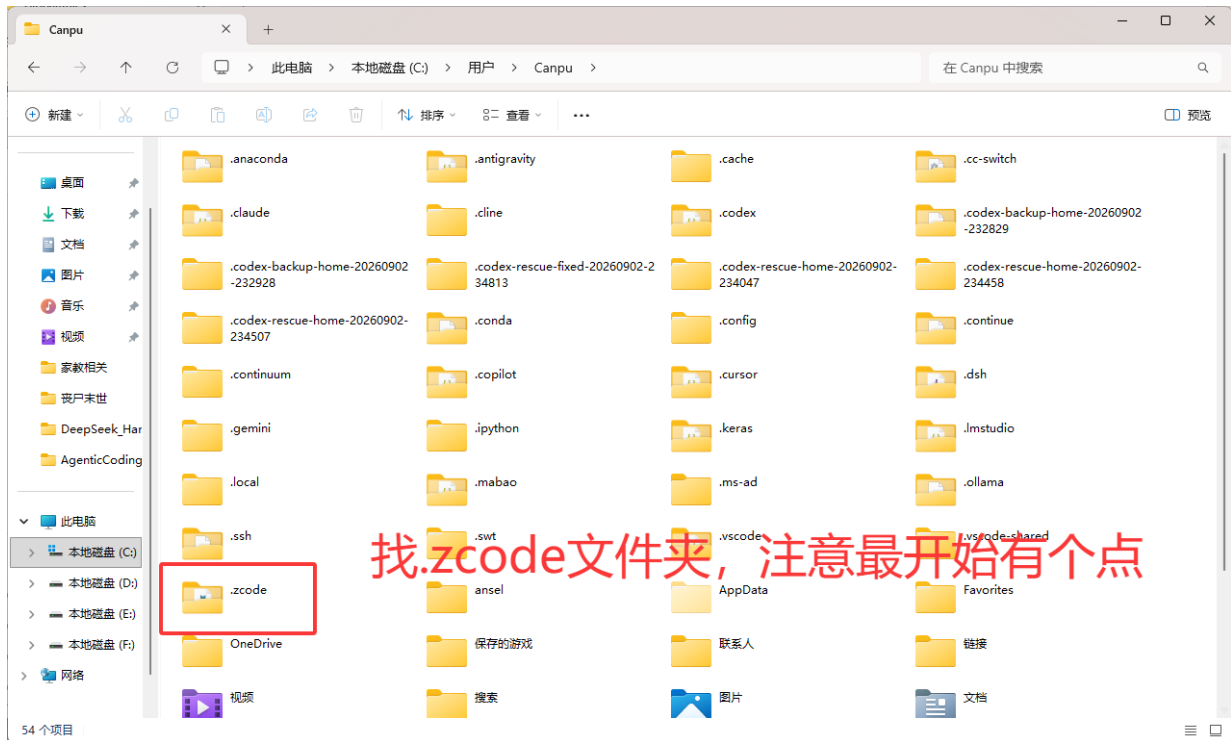


图5 用户目录示例。图中红框演示 .zcode；同一画面也能看到 .codex 与 .dsh。

4.1 Zcode：复制到 .zcode

C:\Users\<你的用户名>\.zcode\

打开解压包中的 GlobalRules，复制其中的全部内容（包括 AGENTS.md 与点号开头的规则目录），粘贴到 .zcode。

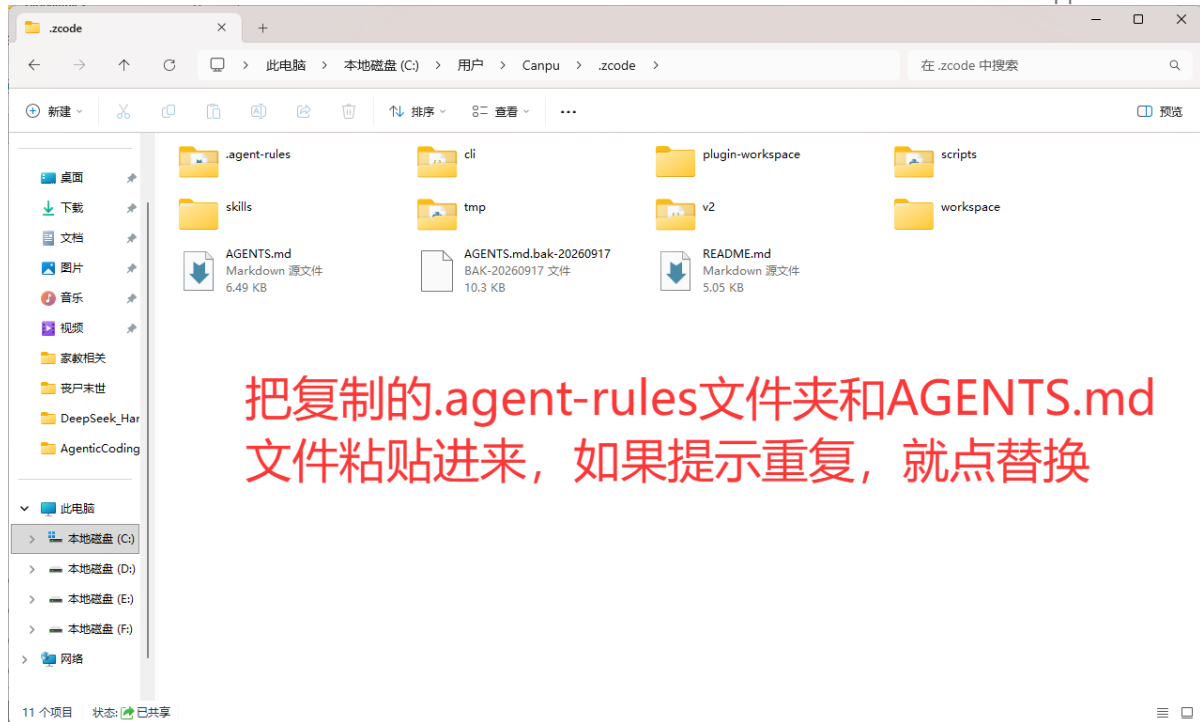


图6 Zcode 示例：把 GlobalRules 的内容合并到 .zcode。

如果 .zcode 中已有你自己的 AGENTS.md 或规则目录，先备份并合并。只有在你确定旧文件就是旧版 PAPOP、且不含自定义内容时，才适合直接替换。

4.2 ChatGPT / Codex：复制到 .codex

C:\Users\<你的用户名>\.codex\

Codex 的用户规则目录默认是用户主目录下的 .codex。PAPOP 的 GlobalRules 应完整合并进去，使全局 AGENTS.md 以及它依赖的相对规则文件都位于同一 Codex Home。

- 如果设置了 CODEX_HOME，请使用 CODEX_HOME 指向的目录，而不是默认 .codex。
- Codex 会优先读取非空的 AGENTS.override.md；若该文件存在，它可能遮蔽普通 AGENTS.md。安装后要实际核验规则来源。
- 在 ChatGPT 桌面端的 Codex 模式中，PAPOP 已经提供项目 AGENTS.md，因此通常不需要再用 /init 生成另一套项目入口；如需生成，请先备份并人工合并。

4.3 DeepSeek Harness：复制到 .dsh

C:\Users\<你的用户名>\.dsh\

DeepSeek Harness 的用户级 AGENTS.md 默认从 DSH_HOME 读取；DSH_HOME 未设置时，默认位置是用户主目录下的 .dsh。把 GlobalRules 的完整内容合并到该目录即可。

- 如果设置了 DSH_HOME，请使用该自定义目录。
- DeepSeek Harness 会把用户级 AGENTS.md 与项目路径上的 AGENTS.md 组合成指令链。
- DeepSeek Harness 当前处于快速迭代阶段；若未来版本改变规则加载方式，应以其官方文档为准。

5. 安装 ProjectRules：复制“里面的内容”到工作区根目录

GlobalRules 只解决全局行为约束。要启用 PAPOP v5 的 Task / Session / Stage、六个账本、Strict Approval 审批和 Checkpoint 恢复，需要再安装 ProjectRules。

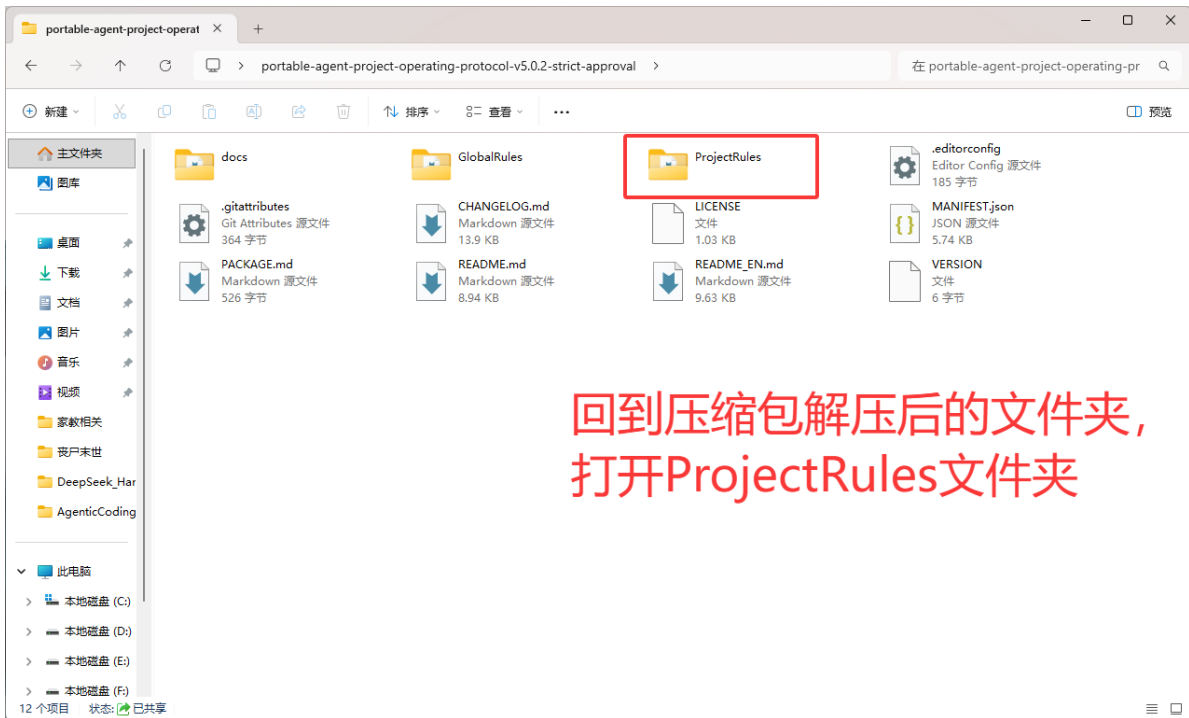


图 7 回到解压目录，打开 ProjectRules。

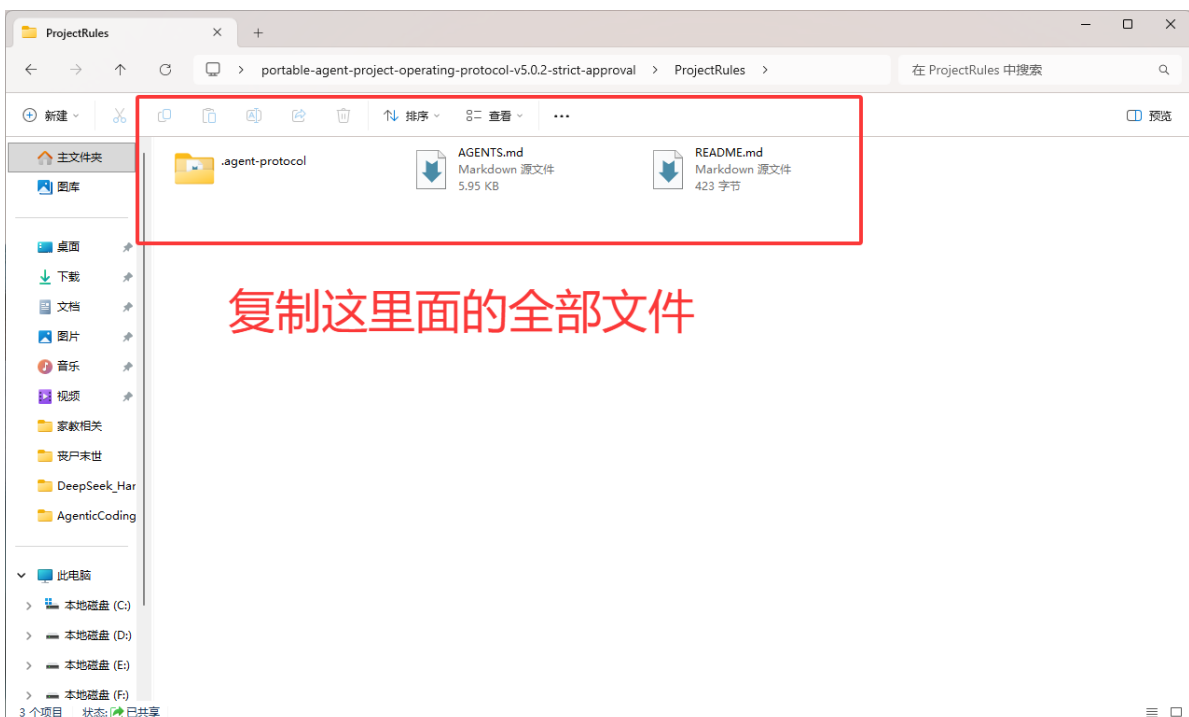


图 8 复制 ProjectRules 里面的全部内容。不要复制 ProjectRules 文件夹本身。

然后进入你实际交给 Agent 工作的“工作区根目录”，直接粘贴。

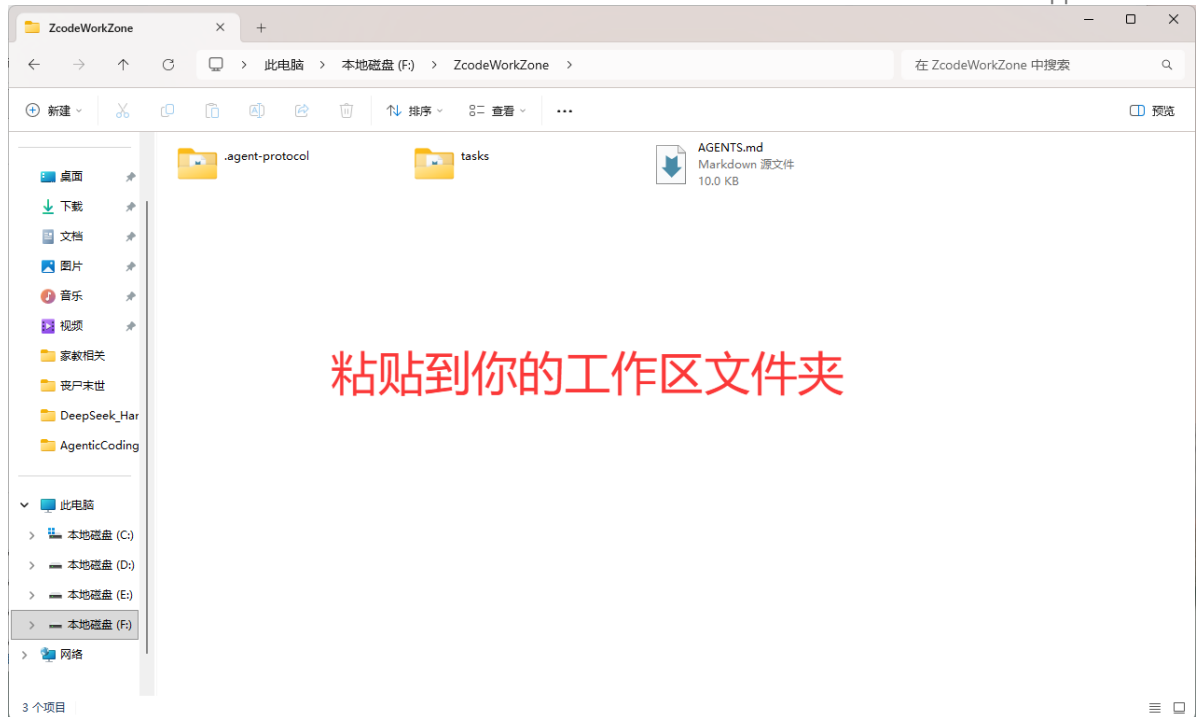


图 9 完成后，工作区根目录应直接出现 AGENTS.md 和 .agent-protocol。

```
<WORKSPACE_ROOT>\
├─ AGENTS.md
├─ .agent-protocol\
├─ tasks\
└─ 你的其他项目文件...
```

错误示例：<WORKSPACE_ROOT>\ProjectRules\AGENTS.md。多套这一层会让 Harness 无法按预期从工作区根加载 PAPOP 入口。

6. 安装后先做三件事：Git 忽略、重开 Session、行为核验

6.1 检查 .gitignore

PAPOP v5 的工作区最少应忽略下面两类运行状态；如果每个 Task 自己又是独立 Git 仓库，该 Task 的 .gitignore 还应忽略 .agent-state/。

```
.agent-work/  
tasks/*/ .agent-state/
```

.agent-state/ 默认不进入 Git，也不会被 PAPOP 自动备份。备份或迁移项目时，必须显式复制包含隐藏目录在内的完整 Task。

6.2 重新开一个全新 Session

规则文件刚复制完成后，不要默认当前旧会话已经重新加载。关闭或新建一次顶层 Session，然后从目标工作区根目录启动 Agent。

6.3 让 Agent 实际核验，而不是只问“你看见了吗”

可以直接发送下面这段检查提示。它的目的不是创建正式 Task，而是确认 Harness 真的读到了正确的规则链：

请实际检查当前加载的全局与工作区规则来源，不要只凭记忆回答。

请说明：

1. 当前工作区使用的是哪一种 PAPOP 模式；
2. Strict Approval 的四个审查点分别是什么；
3. ProjectRules 的入口文件和 .agent-protocol 在哪里；
4. Checkpoint 成功时必须看到什么机器输出；
5. 当前先不要创建正式 Task，也不要修改项目文件。

符合 v5.0.2 Strict Approval 的表现，应能明确识别 Requirements、Goal、Stage Plan、Delivery 四个审查点，并知道只有 Checkpoint 工具输出 CHECKPOINT_COMMITTED 才算检查点提交成功。

如果核验失败

- 确认复制的是 strict-approval ZIP，而不是其他包。
- 确认 GlobalRules 放在当前 Harness 真正使用的用户目录。Codex 检查 CODEX_HOME；DeepSeek Harness 检查 DSH_HOME。
- 确认工作区根直接存在 AGENTS.md 和 .agent-protocol，而不是多套一层 ProjectRules。
- 确认没有旧版或自定义 AGENTS.override.md 把新规则遮蔽。
- 确认重新开了 Session。

7. Strict Approval 日常怎么用

在 Strict Approval 模式里，人类负责确认目标与授权边界；Agent 负责在已批准范围内调查、规划、施工、验证和落盘。最重要的是：审批要明确对应“当前版本”，不要只回复一个含糊的“继续”。

7.1 开始一个新 Task

一个 Task 就是一个完整项目，可以跨很多次对话继续。开始时先说清最终想得到什么、已有材料和不能做什么。

示例：

“我要把这个仓库重构为可发布版本。先调查现状并确认需求；

现在处于 Strict Approval，不要跳过 Requirements、Goal、Stage Plan 和 Delivery 审查点。”

7.2 四个必须由你明确处理的审查点

审查点	你要看什么	建议回复方式
Requirements	需求范围、约束、输入是否理解正确	“需求确认无误；X 需要改成 Y。”
Goal	最终目标、验收条件、边界是否准确	“批准当前 Goal 版本。”
Stage Plan	本阶段步骤、验证方式、授权范围	“批准当前 Stage Plan 版本，仅按该版本执行。”
Delivery	真实产物、diff、测试与机器证据	“验收当前 Delivery” 或明确指出返工项。

Plan R1 的批准不自动授权 R2；返工后新的 Delivery 也需要针对新版本重新验收。审批对象和版本要对应起来。

7.3 Task、Session、Stage 不要混淆

- Task：整个项目。换会话继续同一个项目，不应重复创建 Task。
- Session：一次顶层对话，或上下文压缩 / 换模型 / 交接后的新会话。
- Stage：一个可单独计划、施工、交付、验收的工作周期。一个 Task 可以有多个 Stage。

8. 六个账本、Checkpoint 与上下文压缩

Strict Approval 的价值不只在“停下来问你”，还在于把批准、计划和当前状态写到磁盘。这样对话被压缩、换模型或几天后再继续时，Agent 可以从文件恢复，而不是靠猜。

文件	作用
goal.md	最终目标及修订历史
plan.md	各 Stage 的方案、步骤与审批版本
decisions.md	用户批准、拒绝、授权和改变
current_state.md	当前走到哪里、下一步是什么；核心账本中唯一允许覆写的文件
snapshots.md	连续保存每次 Checkpoint 的完整 Current State
history.md	检索入口，连接 Plan、Decision、Case、Input 与 Snapshot

8.1 准备压缩上下文之前

不要等自动压缩先发生。更稳妥的做法是在一个 Stage 或明显里程碑结束后，主动让 Agent 保存 Checkpoint，并等机器工具确认成功。

建议提示：
“我准备压缩上下文。请先按 PAPOP 保存并核验当前 Checkpoint；
只有确认脚本输出 CHECKPOINT_COMMITTED 后再暂停，不要启动新的工作。”

只有脚本真正输出 CHECKPOINT_COMMITTED 才算保存成功。Agent 自己说“已保存”不能替代机器验证。

8.2 压缩、换模型或新开会话之后

建议提示：
“这是同一个 Task 的新 Session。请先按 PAPOP 恢复流程读取 goal.md 和当前状态，
复述最终目标、当前 Stage、已有批准、未完成项和下一步；
核对实际文件与验证结果无误后再继续，不要重新创建 Task。”

恢复时，状态文件只是入口，不是不可质疑的事实。用户可能在中间改过文件，外部动作也可能已经完成；Agent 仍应核对真实文件、diff、测试和外部状态。

9. 常见问题与容易踩的坑

把 ProjectRules 文件夹整体复制到工作区：错误。应该复制 ProjectRules 里面的 AGENTS.md、.agent-protocol 等内容，让它们直接位于工作区根。

看见同名 AGENTS.md 就直接覆盖：风险很高。已有项目规则、构建命令、保护目录和业务约束都可能被覆盖。先备份，再人工合并。

GlobalRules 文件复制了，但 Agent 说没加载：先重新开 Session，再核对实际 Harness Home。Codex 还要检查 AGENTS.override.md；DeepSeek Harness 要检查 DSH_HOME。

隐藏目录没复制：Windows 可能隐藏点号目录。PAPOP 的规则依赖这些目录，漏掉后即使 AGENTS.md 存在也可能无法完整运行。

Checkpoint 失败但继续压缩：不要继续。未看到 CHECKPOINT_COMMITTED 就当作未提交，先修复并重跑。

换一次聊天就新建一个 Task：错误。同一项目换对话属于新 Session，不是新 Task。

以为 AGENT_COMPLETION=COMPLETE 就等于用户验收：不等于。它只表示 Agent 完成自己的部分；最终是否验收仍由你依据真实交付物和机器证据决定。

依赖 PAPOP 代替备份或权限系统：不行。PAPOP 是行为规则和项目状态机，不替代 Harness 沙箱、操作系统权限和正常备份。

推荐的升级方式

- 升级 PAPOP 前先备份当前工作区和用户级规则。
- GlobalRules 有自定义内容时做差异合并，不要整目录覆盖。
- 旧 Task 的状态机模式会冻结在 Task 中；不要假定替换工作区 AGENTS.md 会自动改变旧 Task 的治理模式。
- 每次升级后都重新做一次规则加载核验和行为检查。

10. 最终检查清单

- ☐ 下载的是 v5.0.2 strict-approval ZIP。
- ☐ GlobalRules 的全部内容已经进入当前 Harness 的用户级目录。
- ☐ ProjectRules 的全部内容已经直接进入工作区根目录。
- ☐ 工作区根能直接看到 AGENTS.md 和 .agent-protocol。
- ☐ 已有规则文件已备份并人工合并，没有误覆盖。
- ☐ .gitignore 已忽略 .agent-work/ 与 tasks/*.agent-state/。
- ☐ 已经新开 Session，并让 Agent 实际说明规则来源与 Strict Approval 四个审查点。
- ☐ 正式使用时，对 Requirements、Goal、Stage Plan、Delivery 都进行明确审批 / 验收。
- ☐ 准备压缩前确认 CHECKPOINT_COMMITTED。
- ☐ 压缩或换会话后继续原 Task，只新增 Session，不重新开 Task。
- ☐ 备份 Task 时包含隐藏的 .agent-state/。

资料与进一步阅读

PAPOP 项目主页

<https://github.com/campus/portable-agent-project-operating-protocol>

PAPOP v5.0.2 Release

<https://github.com/campus/portable-agent-project-operating-protocol/releases/tag/v5.0.2>

PAPOP v5.0.2 README

<https://github.com/campus/portable-agent-project-operating-protocol/blob/v5.0.2/README.md>

PAPOP v5.0.2 INSTALL

<https://github.com/campus/portable-agent-project-operating-protocol/blob/v5.0.2/docs/INSTALL.md>

PAPOP v5.0.2 BEHAVIOR_CHECKS

https://github.com/campus/portable-agent-project-operating-protocol/blob/v5.0.2/docs/BEHAVIOR_CHECKS.md

OpenAI Codex 官方仓库

<https://github.com/openai/codex>

OpenAI: Using Codex with your ChatGPT plan

<https://help.openai.com/en/articles/11369540>

DeepSeek Harness 官方仓库

<https://github.com/deepseek-ai/deepseek-harness>

DeepSeek Harness: agent-instructions

<https://github.com/deepseek-ai/deepseek-harness/blob/master/packages/context/agent-instructions/README.md>

本文中的 PAPP 工作流以 v5.0.2 为准；Codex 与 DeepSeek Harness 的目录和规则加载行为根据 2026-09-17 可核验的官方资料整理。未来客户端若更新，请优先以对应官方文档为准。